Task

A personal book manager.

The app in one picture

- A home page listing your books (fetched from the API).
- Search + filter the list.
- Click a book → its details page.
- Add a new book through a validated form.
- A "favorites" area behind a simple protected route.

Requirements

- Set up the project with Vite + TypeScript.
- Build a `Layout` with a header (app name + nav links) and a main content area.
- Create a reusable `BookCard` component that receives a book via props and renders its title, author, and cover.
- Render a list of `BookCards` from an array. Give each a correct `key`.
- Show an empty state ("No books yet") when the list is empty — using conditional rendering.
- Use composition (`children`) at least once (e.g. a `Card` wrapper used by multiple sections).
- Hold the search text in state; typing updates the visible list live.
- Add a filter (e.g. by genre or read/unread) held in state.
- Derive the filtered list from `books + search + filter` — do **not** store the filtered result in its own state.
- A "mark as read/unread" toggle on each book that updates state.
- If you fetch with `useEffect`, clean it up properly (no state updates after unmount).
- Use `useReducer` somewhere it earns its place (e.g. a multi-field filter panel).
- Provide the current theme (light/dark) via Context; a toggle in the header switches it; any component reads it directly (no prop drilling).
- Write a `useDebounce` hook and use it so search doesn't fire on every keystroke.
- Write a `useLocalStorage` hook and persist the theme (and/or favorites) with it.
- Focus the search input on mount using a ref.
- Build one small compound component (e.g. `Tabs` for "All / Favorites / Read").
- Set up TanStack Query (`QueryClientProvider`).

- Fetch the book list with `useQuery`; show loading and error states.
- Fetch a single book by id with `useQuery` on the details page.
- Add a book with `useMutation`; the list updates after success (invalidate or update the cache).
- Use Axios (a configured instance) as your fetch layer.
- Wrap a risky part of the UI in an Error Boundary.
- Routes: `/` (list), `/books/:id` (details), `/add` (new book form), `/favorites` (protected).
- Use a nested layout route with `Outlet` (header/nav stays, content swaps).
- Read the book id from the route params on the details page.
- Lazy-load at least one route.
- `/favorites` is protected: if a simple "logged-in" flag is false, redirect to `/`.
- After adding a book, navigate programmatically back to the list.
- Build the "add book" form with React Hook Form.
- Validate it with a Zod schema: title required, author required, year a sensible number, genre from a fixed set.
- Show inline validation errors; block submit until valid.

optional

- Optimistic update when toggling read/unread (`useOptimistic` or TanStack optimistic).
- A field array in the form (e.g. multiple tags per book).
- A debounced async validator (e.g. title must be unique — check against the API).
- Persist favorites across reloads.

Suggested `db.json` to start

```
{
  "books": [
    { "id": 1, "title": "The Pragmatic Programmer", "author": "Hunt &
Thomas", "year": 1999, "genre": "tech", "read": true },
    { "id": 2, "title": "Dune", "author": "Frank Herbert", "year":
1965, "genre": "sci-fi", "read": false },
    { "id": 3, "title": "Atomic Habits", "author": "James Clear",
"year": 2018, "genre": "self-help", "read": false }
  ]
```

```
}
```